

MemLab 16 MiB

Manual de usuario

Aprende a asignar, liberar y compactar memoria

Aplicación:	simulador web educativo
Versión documentada:	21 de septiembre de 2026
Memoria simulada:	16 MiB
Instalación:	no requerida

Índice

1. ¿Qué es MemLab?	2
2. Requisitos y apertura	2
2.1. Requisitos	2
2.2. Opción rápida: abrir el archivo	2
2.3. Opción recomendada: servidor local	2
3. Recorrido por la pantalla	2
3.1. Barra superior	2
3.2. Pestañas de método	3
3.3. Indicadores KPI	3
3.4. Mapa de memoria	3
3.5. Tabla de programas	3
3.6. Bitácora	4
4. Primer uso guiado	4
5. Uso de cada método	4
5.1. Particiones fijas iguales	4
5.2. Particiones fijas variables	5
5.3. Particiones dinámicas	5
6. Algoritmos de ajuste en palabras simples	5
7. Crear programas personalizados	5
8. Escenario de fragmentación	6
9. Ejercicios recomendados	6
9.1. Ejercicio A: fragmentación interna	6
9.2. Ejercicio B: comparar ajustes	6
9.3. Ejercicio C: compactación	6
10. Programas incluidos	6
11. Solución de problemas	7
12. Accesibilidad y uso con teclado	7
13. Glosario	8
14. Cierre de la sesión	8

1 ¿Qué es MemLab?

MemLab es un simulador visual para estudiar cómo un sistema operativo reparte la memoria principal entre varios programas. Trabaja con 16 MiB, desde la dirección 0x000000 hasta 0xFFFFF, y permite experimentar sin afectar la memoria real del equipo.

Con MemLab puedes:

- reservar 1, 2 o 4 MiB para el sistema operativo;
- comparar particiones fijas, variables y dinámicas;
- aplicar primer, mejor y peor ajuste;
- cargar o liberar programas y ver dónde quedan ubicados;
- estudiar fragmentación interna y externa;
- compactar la memoria dinámica;
- revisar en la bitácora la explicación de cada decisión.

Importante

La simulación es educativa. No administra la RAM real, no ejecuta los programas listados y no guarda cambios al cerrar o recargar la página.

2 Requisitos y apertura

2.1 Requisitos

Solo se necesita un navegador moderno con JavaScript habilitado. No hacen falta cuenta, conexión a una base de datos, Node.js ni instalación de paquetes. La conexión a Internet se usa únicamente para descargar tipografías; si no está disponible, la aplicación sigue funcionando con fuentes locales.

2.2 Opción rápida: abrir el archivo

Apertura directa

1. Localiza la carpeta del proyecto.
2. Abre `index.html` con doble clic.
3. Espera a que aparezca el mapa de memoria y la tabla de programas.

2.3 Opción recomendada: servidor local

Abre una terminal en la carpeta del proyecto y ejecuta:

```
python3 -m http.server 8000
```

Después visita <http://localhost:8000/>. Para detener el servidor vuelve a la terminal y presiona Ctrl+C.

3 Recorrido por la pantalla

3.1 Barra superior

Muestra el nombre, el rango de direcciones y dos acciones:

Escenario de fragmentación. Prepara automáticamente una distribución con procesos y espacios intercalados para estudiar fragmentación.

Reiniciar. Recrea los programas y la memoria usando la configuración actualmente seleccionada. No necesariamente vuelve al método fijo original.

3.2 Pestañas de método

Las tres pestañas superiores cambian el modo de administración. Al cambiar de método, la memoria se reconstruye y todos los procesos vuelven a la cola.

Método	Idea principal
Estáticas de tamaño fijo	Divide la memoria de usuario en partes iguales. Es fácil de entender, pero desperdicia espacio dentro de cada partición ocupada.
Estáticas de tamaño variable	Usa una tabla de tamaños diferentes. Permite comparar algoritmos de ajuste, pero las particiones siguen sin poder combinarse.
Particiones dinámicas	Crea un bloque del tamaño exacto del programa. Evita fragmentación interna, pero puede dejar huecos externos dispersos.

3.3 Indicadores KPI

Utilización. Porcentaje de la memoria de usuario ocupado por datos útiles de procesos; no incluye el S.O. ni la fragmentación interna.

Fragmentación interna. Espacio sobrante dentro de particiones asignadas. En modo dinámico siempre es cero.

Fragmentación externa. En dinámica, memoria libre fuera del hueco más grande; en estáticas, suma de particiones libres que no pueden unirse.

Mayor bloque libre. Tamaño máximo de un programa que podría entrar de forma inmediata, suponiendo que exista un bloque de ese tamaño.

Procesos. Cantidad cargada frente al total; el subtítulo agrupa trabajos en cola y rechazados.

3.4 Mapa de memoria

La columna representa los 16 MiB completos. La altura de cada bloque es proporcional a su tamaño. Los patrones distinguen:

- sistema operativo;
- bloques ocupados, con un color por proceso;
- particiones o huecos libres;
- fragmentación interna rayada;
- zona no particionada, que no puede utilizarse.

Haz clic en cualquier bloque para seleccionarlo. El panel **Bloque seleccionado** mostrará su rango, tamaño y, si contiene un proceso, el reparto de código, datos, pila y montículo.

3.5 Tabla de programas

Cada fila contiene nombre, PID, barra de segmentos, tamaño total, dirección base, estado y acción disponible. Los estados significan:

cola: el trabajo espera una asignación;
cargado: tiene memoria y puede liberarse;
rechazado: el último intento no encontró un bloque adecuado.

3.6 Bitácora

La bitácora coloca el evento más reciente arriba. Explica cambios de configuración, asignaciones, liberaciones, rechazos, compactaciones e inspecciones. Las marcas **t001**, **t002**, etc. indican el orden lógico, no segundos reales.

4 Primer uso guiado

Al abrir la aplicación se seleccionan 2 MiB para el S.O. y siete particiones fijas de 2 MiB. Se cargan Editor de texto, Compilador C, Terminal y Hoja de cálculo.

1. Examinar el estado inicial

1. Observa los cinco KPI.
2. Haz clic en el bloque del Editor de texto.
3. Comprueba el rango, el registro base y sus cuatro segmentos.
4. Identifica la franja de fragmentación interna dentro de su partición.

Qué observar

El Editor ocupa 1 MiB dentro de una partición de 2 MiB; queda 1 MiB de fragmentación interna que ningún otro proceso puede aprovechar.

2. Cargar y liberar

1. Busca un programa en estado **cola**.
2. Presiona **Cargar**.
3. Revisa su base y la nueva línea de bitácora.
4. Presiona **Liberar** y observa cómo vuelve a cola.

3. Reiniciar el ejercicio

Presiona **Reiniciar**. El catálogo y las asignaciones iniciales se recrean con el método, el S.O. y las opciones que estén activos en ese momento.

5 Uso de cada método

5.1 Particiones fijas iguales

1. Selecciona **Estáticas de tamaño fijo**.
2. Elige 1, 2 o 4 MiB para el S.O.
3. Elige entre 2 y 16 particiones según las opciones disponibles.
4. Lee el tamaño calculado debajo del selector.
5. Carga programas uno por uno o usa **Cargar la cola completa**.

El algoritmo de ajuste no se puede elegir en este modo: como todas las particiones miden lo mismo, se toma la primera libre capaz de contener el proceso. Si un programa supera el tamaño de una partición, no podrá cargarse aunque la suma de varias particiones libres sea suficiente.

5.2 Particiones fijas variables

1. Selecciona **Estáticas de tamaño variable**.
2. Revisa los chips que indican tamaños en KiB.
3. Para eliminar una partición, presiona la $\times$ de su chip.
4. Para añadir una, escribe al menos 64 KiB y presiona **Añadir**.
5. Escoge primer, mejor o peor ajuste.
6. Carga trabajos y compara las decisiones en la bitácora.

Los tamaños añadidos se ordenan automáticamente de menor a mayor. La interfaz no permite añadir una partición si la suma excede la memoria de usuario disponible. Sin embargo, si primero defines una tabla grande y después aumentas el S.O., algunas entradas finales pueden no construirse; revisa siempre el mapa y el texto de suma.

5.3 Particiones dinámicas

1. Selecciona **Particiones dinámicas**.
2. Escoge un algoritmo de ajuste.
3. Carga varios programas.
4. Libera procesos intermedios para producir huecos separados.
5. Observa **Frag. externa** y **Mayor bloque libre**.
6. Presiona **Compactar ahora** y compara el mapa antes/después.

Activa **Compactar automáticamente al fallar una asignación** si quieres que el simulador compacte cuando el total libre alcanza, pero ningún hueco individual es suficientemente grande. No compacta si falta memoria total.

6 Algoritmos de ajuste en palabras simples

Algoritmo	Cómo decide	Qué conviene observar
Primer ajuste	Recorre los bloques en orden de dirección y elige el primero que alcance.	Rapidez conceptual y dependencia del orden.
Mejor ajuste	Compara todos los candidatos y usa el más pequeño posible.	Resto inmediato pequeño y aparición de huecos diminutos.
Peor ajuste	Compara todos y usa el bloque más grande.	Resto grande potencialmente reutilizable.

No existe un algoritmo universalmente mejor: el resultado depende del orden y tamaño de los trabajos. MemLab permite repetir el mismo escenario con distintas políticas para comparar.

7 Crear programas personalizados

Abre el desplegable **Añadir un programa a la cola**. Completa:

- nombre, con máximo visual de 28 caracteres en el formulario;
- código, datos, pila y heap en KiB;
- como mínimo 8 KiB de código; los otros segmentos pueden quedar en cero si el navegador acepta el valor introducido.

Presiona **Añadir a la cola**. El tamaño final es la suma de los cuatro segmentos. No se acepta un proceso mayor que la memoria de usuario actual.

Datos de práctica

Usa nombres de texto normal. Esta versión representa parte de la bitácora como HTML; no introduzcas etiquetas ni código en el nombre del programa.

8 Escenario de fragmentación

El botón **Escenario de fragmentación** realiza automáticamente lo siguiente:

1. restaura los ocho programas de ejemplo;
2. reconstruye la memoria con la configuración activa;
3. intenta cargarlos en orden;
4. libera los procesos cargados en posiciones alternas;
5. muestra cuánto queda libre, cuántos bloques hay y cuál es el mayor;
6. sugiere intentar el trabajo pendiente más grande.

En modo dinámico, este escenario es la forma más rápida de estudiar el contraste entre memoria libre total y memoria contigua. Prueba a cargar el proceso sugerido, compacta y vuelve a intentarlo.

9 Ejercicios recomendados

9.1 Ejercicio A: fragmentación interna

1. Usa 2 MiB de S.O. y siete particiones fijas.
2. Reinicia.
3. Inspecciona Editor, Terminal y Hoja de cálculo.
4. Suma las franjas de fragmentación interna.

Resultado esperado inicial: 3 136 KiB de fragmentación interna entre los cuatro procesos precargados.

9.2 Ejercicio B: comparar ajustes

1. Selecciona particiones variables y reinicia.
2. Anota dónde se ubica cada trabajo con primer ajuste.
3. Repite desde el mismo estado con mejor ajuste y luego con peor ajuste.
4. Compara mayor bloque libre, rechazos y fragmentación interna.

9.3 Ejercicio C: compactación

1. Selecciona dinámica y desactiva compactación automática.
2. Ejecuta el escenario de fragmentación.
3. Anota libre total, mayor hueco y fragmentación externa.
4. Compacta manualmente.
5. Comprueba que los procesos quedan juntos al inicio y un solo hueco al final.

10 Programas incluidos

Programa	Tamaño	Comentario práctico
Editor de texto	1 024 KiB	Encaja exactamente en 1 MiB y deja 1 MiB en una partición de 2 MiB.
Compilador C	2 048 KiB	Encaja exactamente en la partición fija predeterminada.
Navegador web	3 840 KiB	No cabe en una partición fija predeterminada.
Motor de base de datos	3 072 KiB	Útil para tablas variables y huecos medianos.
Terminal	448 KiB	Produce fragmentación interna alta en bloques grandes.
Render 3D	6 144 KiB	Trabajo más grande; exige una partición o hueco de 6 MiB.
Servicio de red	768 KiB	Permite rellenar espacios pequeños.
Hoja de cálculo	1 536 KiB	Deja 512 KiB en una partición de 2 MiB.

11 Solución de problemas

Situación	Qué hacer
La página aparece sin datos	Confirma que JavaScript esté habilitado y que <code>js/script.js</code> conserve su ruta relativa. Abre la consola del navegador para buscar errores.
No se ven los estilos	Verifica que exista <code>css/style.css</code> . Si usas servidor, confirma que responda con estado 200.
Un programa es rechazado	Lee la bitácora. Puede ser mayor que cada partición, faltar memoria total o existir fragmentación externa.
Hay memoria libre pero no carga	Revisa Mayor bloque libre . En dinámica, compacta; en estáticas, los bloques no se pueden unir.
Compactar no cambia nada	Solo aplica a dinámica. También puede no haber procesos o ya estar contiguos.
Cambiar método borró asignaciones	Es el comportamiento esperado: cada disposición tiene límites diferentes y los procesos vuelven a cola.
Reiniciar no volvió a siete particiones	Reiniciar conserva la configuración activa; selecciona primero el método y valores deseados.
Las letras se ven diferentes	Las fuentes web no cargaron; la aplicación usa una fuente alternativa y sigue funcionando.

12 Accesibilidad y uso con teclado

Los botones y campos se pueden recorrer con **Tab**; usa **Enter** o barra espaciadora para activarlos. El foco visible indica el control actual. Las pestañas y algoritmos anuncian su estado mediante **aria-pressed**, y la bitácora informa cambios con una región activa cortés. El diseño sigue el tema claro u oscuro del sistema y evita transiciones si está habilitada la reducción de movimiento.

En pantallas estrechas, el mapa pasa encima de los paneles laterales. Las tablas pueden desplazarse horizontalmente.

13 Glosario

Base. Primera dirección física asignada a un proceso o bloque.

Compactación. Reubicación de procesos para reunir el espacio libre.

Fragmentación externa. Espacio libre dividido en regiones separadas.

Fragmentación interna. Espacio desperdiciado dentro de una partición ocupada.

Heap o montículo. Segmento usado para memoria dinámica del programa.

Hueco. Región libre en el método dinámico.

KiB. 1024 bytes.

MiB. 1024 KiB o 1 048 576 bytes.

PID. Identificador numérico interno del proceso simulado.

Pila. Segmento asociado a llamadas, variables locales y control de ejecución.

Registro límite. Tamaño máximo válido relativo a la base del proceso.

14 Cierre de la sesión

No existe una operación de guardado. Si necesitas conservar resultados, toma una captura o registra manualmente KPI y bitácora antes de cerrar. Detén el servidor local con **Ctrl+C** si lo iniciaste desde la terminal.